

Activitat 1

En esta actividad procederá a crear y documentar la base de datos que utilizaremos durante toda la actividad.

Activitat 1.1

Inicie MongoDB Compass y cree una nueva base de datos llamada **sample_supplies** Dentro de esta crea la colección [sales](#)

Create Database x

Database Name

Collection Name

Una vez creadas ambas importa los datos del fichero JSON adjunto al enunciado de esta EAC dentro de la colección mencionada anteriormente

Activitat 2

Responda a las siguientes preguntas trabajando siempre desde la Shell copiando en formato texto los pedidos usados así como la captura de la ejecución de los mismos

Para la realización de las siguientes preguntas no será necesario que use el framework de agregación de MongoDB

Activitat 2.1

Lance una consulta que indique cuántas ventas hay almacenadas en la colección **sales**

```
db.sales.countDocuments()
```

```
> db.sales.countDocuments()  
< 5000  
sample_supplies> |
```

Activitat 2.2

¿Cuántas ventas se han realizado en tiendas de **London** o **New York**?

```
db.sales.countDocuments({
```

```
{
```

```
  $or: [
```

```
    {storeLocation:"London"},
```

```
    {storeLocation: "New York" }
```

```
  ]
```

```
})
```

```
> db.sales.countDocuments(  
  {  
    $or: [  
      {storeLocation:"London" },  
      {storeLocation: "New York" }  
    ]  
  })  
< 1295  
sample_supplies> |
```

Activitat 2.3

Responde a la pregunta anterior usando un operador diferente

```
db.sales.countDocuments({
```

```
  {storeLocation: {
```

```
    $in: ["London", "New York"]
```

```
  }
```

```
})
```

```
> db.sales.countDocuments(  
  {storeLocation: {  
    $in: ["London", "New York"]  
  }  
})  
< 1295
```

Activitat 2.4

¿Cuántas ventas se han realizado en las ciudades de **Denver** o **London** cuando el método de compra ha sido el teléfono **Phone**

```
db.sales.countDocuments({
```

```
  storeLocation: {$in: ["Denver", "London"]}, purchaseMethod: "Phone"
```

```
})
```

```
> db.sales.countDocuments({  
  storeLocation: {$in: ["Denver", "London"]}, purchaseMethod: "Phone"  
})  
< 275
```

Activitat 2.5

Si se fija en la colección **sales** hay una clave **items** que es un array de objetos y que contiene el número de productos comprados

Queremos saber cuántas ventas se han realizado en **New York** en las que se haya vendido únicamente un solo producto

```
db.sales.countDocuments({
```

```
  {storeLocation: "New York", items: {$size: 1}}})
```

```
> db.sales.countDocuments(  
  {storeLocation: "New York", items: {$size: 1}})  
< 39
```

Activitat 2.6

Queremos saber ahora cuántas ventas se han hecho de **laptops** de entre 900 y 950 euros

```
db.sales.countDocuments({
```

```
  {items: {
```

```
    $elemMatch: {
```

```
      name: "laptop",
```

```
      price: {$gte: 900, $lte: 950}
```

```
    }
```

```
  }
```

```
})
```

```
> db.sales.countDocuments(  
  {items: {  
    $elemMatch: {  
      name: "laptop",  
      price: {$gte: 900, $lte: 950}  
    }  
  }  
})  
< 100
```

Activitat 2.7

Cuántas ventas se han hecho de **laptops** de entre 900 y 950 euros en tiendas de **New York**

```
db.sales.countDocuments({
```

```
  {storeLocation: "New York",
```

```
  items: {
```

```
    $elemMatch: {
```

```
      name: "laptop",
```

```
      price: {$gte: 900, $lte: 950}
```

```
    }
```

```
  }
```

```
})
```

```
> db.sales.countDocuments(  
  {storeLocation: "New York",  
  items: {  
    $elemMatch: {  
      name: "laptop",  
      price: {$gte: 900, $lte: 950}  
    }  
  }  
})  
< 11
```

Activitat 2.8

Para documentos de ventas con algún artículo con un precio de venta mayor a 1598 euros muestra sólo

- el nombre
- precio
- cantidad de los artículos vendidos
- la localización de la tienda
- el correo electrónico del cliente

Ordene los documentos mostrados alfabéticamente por email de los clientes

```
db.sales.find(
```

```
  {"items.price": {$gt: 1598}},
```

```
  {"items.name": 1, "items.price": 1, "items.quantity": 1, storeLocation: 1, "customer.email": 1, _id: 0}
```

```
).sort({"customer.email": 1})
```

```
> db.sales.find(
  {"items.price": {$gt: 1598}},
  {"items.name": 1, "items.price": 1, "items.quantity": 1, storeLocation: 1, "customer.email": 1, _id: 0}
).sort({"customer.email": 1})
< {
  items: [
    {
      name: 'notepad',
      price: Decimal128('36.16'),
      quantity: 4
    },
    {
      name: 'printer paper',
      price: Decimal128('35.84'),
      quantity: 2
    },
    {
      name: 'notepad',
      price: Decimal128('18.42'),
      quantity: 4
    },
    {
      name: 'envelopes',
      price: Decimal128('5.97'),
      quantity: 2
    },
    {
      name: 'binder',
      price: Decimal128('25.98'),
```

Activitat 2.9

Muestra

- la botiga
- el correo del cliente

para aquellas 5 ventas de laptops más caras

No queremos que aparezca el id

Sabemos que los laptops son los artículos más caros con diferencia de entre los que vendemos.

```
db.sales.find(  
  {"items.name": "laptop"},  
  {storeLocation: 1, "customer.email": 1, "items.price": 1, _id: 0}  
) .sort({"items.price": -1}).limit(5)
```

```
> db.sales.find(  
  {"items.name": "laptop"},  
  {storeLocation: 1, "customer.email": 1, "items.price": 1, _id: 0}  
) .sort({"items.price": -1}).limit(5)  
< {  
  items: [  
    {  
      price: Decimal128('73.22')  
    },  
    {  
      price: Decimal128('32.74')  
    },  
    {  
      price: Decimal128('10.3')  
    },  
    {  
      price: Decimal128('1599.49')  
    },  
    {  
      price: Decimal128('6.61')  
    },  
    {  
      price: Decimal128('27.35')  
    },  
    {  
      price: Decimal128('39.09')  
    },  
    {  
      price: Decimal128('28.2')  
    }  
  ]  
}
```

Activitat 2.10

Registra una nueva venta realizada hoy en la tienda de Barcelona

El cliente es

- una señora
- de 28 años
- correo `marta@test.com`

Ha comprado

- un único producto
- un `"laptop"`
- precio `1100.50` euros
- etiqueta `"electronics"`
- cantidad `1`

No ha utilizado cupón y el método de compra es `"In store"`

La fecha del campo `saleDate` será la del día que ejecute el pedido de inserción

```
db.sales.insertOne(  
  {saleDate: new Date(),  
  items: [{name: "laptop", tags: ["electronics"], price: 1100.50, quantity: 1}],  
  storeLocation: "Barcelona",  
  customer: {gender: "F", age: 28, email: "marta@test.com"},  
  couponUsed: false,  
  purchaseMethod: "In store"  
})
```

```
> db.sales.insertOne(  
  {saleDate: new Date(),  
  items: [{name: "laptop", tags: ["electronics"], price: 1100.50, quantity: 1}],  
  storeLocation: "Barcelona",  
  customer: {gender: "F", age: 28, email: "marta@test.com"},  
  couponUsed: false,  
  purchaseMethod: "In store"  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('6a187ab7496f402236f3f129')  
}
```


Activitat 2.11

Realice una búsqueda para demostrar que la inserción se ha producido efectivamente

```
db.sales.find({"customer.email": "marta@test.com"})
```

```
> db.sales.find({"customer.email": "marta@test.com"})
< {
  _id: ObjectId('6a187dee496f402236f3f12c'),
  saleDate: 2026-05-28T17:39:58.706Z,
  items: [
    {
      name: 'laptop',
      tags: [
        'electronics'
      ],
      price: 1100.5,
      quantity: 1
    }
  ],
  storeLocation: 'Barcelona',
  customer: {
    gender: 'F',
    age: 28,
    email: 'marta@test.com'
  },
  couponUsed: false,
  purchaseMethod: 'In store'
}
```

Activitat 2.12

El anterior cliente de la tienda de Barcelona con el correo marta@test.com ha decidido añadir un producto de última hora a su compra

- un "mouse"
- de 25.00 €
- con el tag "accessory"

Realice la modificación en su compra.

```
db.sales.updateOne(  
  {"customer.email": "marta@test.com"},  
  {  
    $push: {  
      items: {name: "mouse", tags: ["accessory"], price: 25.00, quantity: 1}  
    }  
  }  
)
```

```
> db.sales.updateOne(  
  {"customer.email": "marta@test.com"},  
  {  
    $push: {  
      items: {name: "mouse", tags: ["accessory"], price: 25.00, quantity: 1}  
    }  
  }  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

Lance una consulta que demuestre que ésta se ha producido efectivamente:

```
db.sales.find({"customer.email": "marta@test.com"})
```

```
> db.sales.find({"customer.email": "marta@test.com"})
< {
  _id: ObjectId('6a187dee496f402236f3f12c'),
  saleDate: 2026-05-28T17:39:58.706Z,
  items: [
    {
      name: 'laptop',
      tags: [
        'electronics'
      ],
      price: 1100.5,
      quantity: 1
    },
    {
      name: 'mouse',
      tags: [
        'accessory'
      ],
      price: 25,
      quantity: 1
    }
  ],
  storeLocation: 'Barcelona',
  customer: {
    gender: 'F',
    age: 28,
    email: 'marta@test.com'
  },
}
```

Activitat 2.13

Borre el documento insertado en el punto anterior.

```
db.sales.deleteOne({"customer.email": "marta@test.com"})
```

```
> db.sales.deleteOne({"customer.email": "marta@test.com"})
< {
  acknowledged: true,
  deletedCount: 1
}
```

Lance una consulta de búsqueda que demuestre que efectivamente se ha borrado este:

```
db.sales.find({"customer.email": "marta@test.com"})
```

```
> db.sales.find({"customer.email": "marta@test.com"})
<
sample_supplies> |
```